脆弱性情報の上流は安定していない

公開前検査による116日間の全数観測

篠原 俊一・中岡 典弘・神戸 康多(フューチャー株式会社)

コンピュータセキュリティシンポジウム 2026 / OWS 2026

前回のあらすじ: 履歴管理された脆弱性DB基盤

- OSS 脆弱性スキャナ Vuls のエコシステム
  - 40+ データソース → 収集(fetch) → 変換(extract) → DB構築 → 6時間ごとに公開
- 昨年(CSS 2025)「広範な脆弱性情報の統合管理と履歴追跡」
  - 全工程を Git + digest で履歴管理、任意時点を追跡・再現できる
- 本稿はその続編: 記録できる → 実際にどう変化しているかを測る

暗黙の仮定: 「上流フィードは安定している」

- スキャナの検知結果は上流のアドバイザリ/フィードをほぼそのまま反映する
- DB の自動更新は「上流は概ね安定、変化は新規脆弱性の追加が中心」を暗黙に仮定
- この仮定はどの程度正しいのか?

> 答えを 116 日間の本番運用データで示すのが本発表

動機: 2026年3月の2つのインシデント

	インシデント1	インシデント2
上流	Red Hat VEX	Ubuntu CVE Tracker
何が	不正データ×形式変更で空データ	DB のマーカを旧スキャナが解釈できず
影響	redhat:9 の検知 51.9% が消えた(FN)	検知が +106.0% に倍増(FP)
共通	ビルドは成功、公開も成功	同左

- CI が確かめるのはビルドの成否であって、生成物がデータとして正しいかではない

公開前検査 (diff guard)

図: パイプライン全体図(論文 図1 を SVG 化)
上流 → raw(Git) → extracted(Git) → 候補DB —→ [検査 Dn/Do/DB] → PASS: 公開 / FAIL: 保留 → 人間レビュー → promote

- 公開候補 DB を直前の公開版(baseline)と自動比較
- 変動率が閾値を超えたら公開を保留し、人間の確認を強制
- 検査は変更の正誤を判定しない(平常帯から外れた変更を人間に見せる)

3つのチェック

略記	何を比較	捕まえるもの
Dn	最新スキャナの検知結果(fixture 75本)	検知の消失・激増
Do	固定した旧スキャナの検知結果	旧版との非互換(インシデント2型)
DB	DB 構造(leaf Criterion 約330万)	fixture 外も含む構造変化

- 変動率 = (追加+削除) / baseline、対象ごとに1行、1行でも超過なら FAIL
- 3つ全部を実行してから集約(先に落ちたチェックが後続を隠さない)

FAIL したらどうなるか

- 候補はタグなし digest のまま GHCR に残る(公開されない・事後検証可能)
- オペレータがレポートを確認 → 正当なら専用 workflow で promote(タグ付け)
- 実行者・digest・タグ・判断材料が履歴に残る = 監査証跡付きの手動承認
- promote の判断と実行は人間に限定(AI エージェントには許可しない)

閾値は成立するのか: 導入前ベンチマーク

図: 導入前ベンチマーク(論文 図2、figs/fig2 を流用)

- 導入前に 37 日分の DB で連続1日ペア 35 組を実測
- 日常変動は中央値 0.3%、最大 6.2%
- 10% を超えた 2 組はどちらも実在の上流イベント(Red Hat VEX 65.1% / SUSE OVAL 16.1%)
- 正常帯と異常イベントが桁で分離 → 単純な閾値検査が成立
- 反復的な正当変動は対象別 override + 約2か月ごとの再導出(grooming)

測定: 116日間・943 run の全数調査

- 2026-04-23 〜 08-16、安定版/開発版の全 run を GitHub API で収集
- FAIL 中は baseline が止まり同じ差分が 6 時間ごとに再検出される
  - 連続 FAIL を束ねる: fail sequence 104 件
  - 2系統統合 + ソース単位に分解: source-episode 69 件(原因分布の単位)
- 24h 集約規則(12h なら 77 件 / 48h なら 62 件)

原因帰属: 履歴で工程を消していく

1. DB構築工程: DB メタデータのバージョン比較
2. 収集・変換工程: 当該ソースのコード履歴を走査
3. 収集対象設定: seed リスト・対象ソースの変更を走査
4. 上流: raw の Git 履歴から件数・ID が一致する変更実体(smoking gun)

- 全 69 episode を帰属、「原因不明」は残らなかった
- 全工程の履歴管理が帰属の成立条件(昨年の基盤の実証)

結果: 原因分布

判定	件数	代表例
上流: 正当な変更	55	月次パッチ等
上流: 一時的障害	2	CVRF 消失
上流: 恒久的品質イベント	2	errata 再編
変換コード(バグ / 意図的変更)	1 / 7	非決定的出力 / CPE 系列
収集対象設定	2	収集起点 ID 追加

- 上流由来 59 件(平均約2日に1件)、自前の処理由来 10 件

116日間のタイムライン

図: source-episode 69件のタイムライン(論文 図3、figs/fig3 を流用)

上流は消える(可用性)

- Microsoft CVRF の月次データは消え、戻り、また消えた
  - 2026年6月分のファイル数: 1273 → 0 → 1273 → 0 → 1278
  - 同じ月のデータが 6 月と 7 月の 2 度消失
- API は正常応答のまま。収集処理の成功とデータの可用性は同義ではない

上流は書き換わる(完全性)

- AlmaLinux: errata が縮退(279→60 件、CVE の 87% が消失)。上流は仕様どおり
- Cisco openVuln: 更新日時もバージョンも変えずマッピングだけ日次で書き換え(告知なし)
- VulnCheck: 全年代 CVE へ CPE 一括付与 → 拡充と検知退行が同時に
- 正常か異常かの二分は難しい。平常帯から外れた変更を履歴とともに人間へ

正当でも桁違いに動く(変動規模)

図: FAIL 1,465行の変動率分布(論文 図4、figs/fig4 を流用)

- 頂点は fedora:46 の 3433.3%(小さい baseline)、AlmaLinux 428.3%
- 日常変動の多くは 5〜20% → 構造的イベントと概ね桁で分離

検査は事故を防げたか: 止めた3件

- CVRF 全消失 ×2: 約 1,270 CVE 分の検知消失に相当する DB を公開せず破棄
- 変換コードの非決定性バグ ×1:
  - DB チェックが 274.5% の drift を検出
  - extracted 23,163 ファイル変化 vs raw はわずか 266 → 自前のバグと判明
  - コンパイルもテストも通過するバグが出力の構造差分で初めて見えた

防ぎ切れなかった2件 → 最弱環は承認側

- AlmaLinux 縮退: FAIL 滞留の解消時に縮退候補を promote してしまい数時間公開
- VulnCheck: ecosystem 一括の変動率(28.9%)が実態(単独 189.6%)を希釈 → promote
- 2件とも検出と保留までは成功。残る問題はレポートの分解能と人間の承認判断
- → ソース単位化、削除方向の自動ブロック、複数名承認

運用コストの実態

- 943 run 中 420 run(44.5%)が FAIL、手動 promote は 0.79件/日(ほぼ毎日1回)
- FAIL の 50% は土日 = 上流ではなく人間の応答時間の信号(週末滞留)
- 保留中は直前版を配り続ける = 新規 CVE の反映遅延という別のリスク
- 計算時間は DB 1分 / 検知 9分 → 6時間周期の CI に常設できる

まとめ

- 上流は消え、戻り、遡及的に書き換わり、アナウンスなく再編される。大半は正当な変更
- 全工程の履歴管理があれば、発動の全件を根拠付きで原因帰属できる
- 検査が保証するのは自動保留まで。最終判断は人間に残る(承認側の設計が次の課題)

公開しています

論文・データ(run 表、69 episode 判定表、集計スクリプト)
github.com/vulsio/css2026-diff-guard

検査の実装: vuls2 / 閾値設定: vulsio/vuls-data-db

ご質問・追試歓迎です